

Université Paris Cité
Master's Degree – Computer Science

Scalable and Robust Music Identification with Audio Embeddings, Vector Search and Fingerprinting

Project Report
Academic Year: 2025–2026

Students: AIT MOKHTAR Clara
AYDIN Maria
GOURMELEN Thomas
TAN Vincent
Course: Big Data
Teacher: PALPANAS Themis
Repository: github.com/thmsg018/Shazam
Date: April 17, 2026

Abstract

This report presents a complete Big Data pipeline for music identification from short audio excerpts on a final reference corpus of 4 257 tracks from 1 639 artists. The project combines automated dataset construction, metadata enrichment, audio preprocessing, learned embeddings, vector search, and Shazam-inspired fingerprinting in order to identify a track from either a clean file or a real microphone recording. The main contribution of the work is not only the implementation of an identification model, but the design of a scalable retrieval system integrating data acquisition, storage, indexing, validation, and evaluation. The system uses a hybrid data architecture with ChromaDB for persistent embeddings, FAISS for fast nearest-neighbor retrieval, SQLite for fingerprints, and Parquet for metadata. Experimentally, we distinguish a real-query evaluation suite and complementary robustness ablations. On the 59-query real suite, Stage 2 re-ranking raises Top-1 accuracy from 22.0% to 72.9%. On a complementary robustness benchmark used for controlled ablations, RIR augmentation raises Top-1 accuracy from 57.1% to 85.7%. Across both protocols, the most important practical gains come from query preprocessing and from room-impulse-response-based augmentation of the reference database.

Keywords: music identification, audio retrieval, vector search, FAISS, embeddings, fingerprinting, Big Data

Contents

1	Introduction	1
2	Related Work	1
2.1	Classical Audio Fingerprinting	1
2.2	Learned Audio Embeddings	2
2.3	Large-Scale Similarity Search	2
2.4	Robustness and Signal Enhancement	2
2.5	Positioning of Our Work	3
3	Problem Formulation	3
3.1	Segmentation and Representation	3
3.2	Formalizing Degradations	3
3.3	Stage 1: Segment-Level Vector Retrieval	3
3.4	Stage 2: Fingerprint-Based Re-Ranking	4
4	Solution	4
4.1	End-to-End Pipeline Overview	4
4.2	Building the Reference Database	4
4.3	Metadata Enrichment	4
4.4	Audio Processing and Representations	5
4.5	Fingerprinting Stage	6
4.6	Data Management and Storage Architecture	7
4.7	Query Pipeline	7
4.8	Robustness Through RIR Augmentation	8
5	Experiments	8
5.1	Experimental Setup	8
5.2	Overall Evaluation	9
5.3	Studio Versus Microphone Queries	10
5.4	Effect of Query Duration	10
5.5	Stage 1 Versus Stage 2	10
5.6	Microphone Condition Analysis	10
5.7	Complementary Robustness Ablations	10
5.8	Discussion	11
5.9	Evaluation Scope and Current Limits	11
6	Conclusion	12
A	Use of Artificial Intelligence	12
B	References	13
C	Basic Usage and Reproducibility	14
D	User Interfaces	14
E	Visual Appendix	14

1 Introduction

Music identification consists in retrieving, from a short audio excerpt, the corresponding track in a reference database. Although this problem is widely known through consumer applications such as Shazam, it remains an interesting research and engineering task because it lies at the intersection of signal processing, multimedia information retrieval, vector search, and data management. In practice, a useful system must do much more than compare two signals: it must acquire and clean data, compute compact representations, store them in suitable structures, answer queries efficiently, and remain robust when the observed signal is degraded by real recording conditions.

The industrial scale of the problem also shows why music identification is difficult. In November 2024, Apple reported that Shazam had surpassed 100 billion song recognitions since launch and served more than 300 million monthly active users [1]. These figures illustrate that real-world music recognition systems must simultaneously support very large usage volumes and very large song catalogs, while remaining fast and reliable under noisy and heterogeneous recording conditions.

The goal of our project is to design a complete end-to-end pipeline for music identification. The system accepts either a clean audio file or a microphone recording and returns the most likely tracks from a reference collection. The project therefore goes beyond a simple demonstration interface: it includes automated track acquisition, metadata enrichment, segmentation, embedding computation, fingerprint extraction, hybrid storage, vector indexing, retrieval, and experimental evaluation.

This problem is interesting for three main reasons. First, it is concrete and easy to evaluate quantitatively. Second, it naturally raises Big Data questions: how to construct and maintain the database, how to index tens or hundreds of thousands of vectors, and how to choose the right storage system for each data type. Third, it provides a direct comparison between two important methodological families: learned audio embeddings and Shazam-like fingerprinting.

The problem is also difficult. Clean excerpts are usually easy to match, but real queries are often degraded by background noise, reverberation, codec compression, microphone coloration, and loudness variations. Moreover, each reference track is segmented into multiple overlapping windows, which quickly increases the number of indexed items. Retrieval must therefore remain scalable while preserving accuracy.

The main contributions of the project are the following:

- a complete data pipeline for automatic track acquisition, processing, storage, and retrieval;
- a hybrid data architecture combining ChromaDB, FAISS, SQLite, and Parquet;

- a two-stage identification pipeline based on embeddings and fingerprint re-ranking;
- a detailed experimental study of robustness under realistic degradations;
- a practical improvement of microphone-query performance through query preprocessing and room impulse response augmentation.

2 Related Work

The papers studied during the project can be grouped into four main families: classical audio fingerprinting, learned audio embeddings and music representations, large-scale similarity search, and robustness or signal enhancement. Rather than citing only a few emblematic papers, we summarize the full set of readings used by the group and explain how each family relates to the implemented system.

2.1 Classical Audio Fingerprinting

The first family covers classical fingerprinting systems for exact music identification. Wang’s industrial-strength Shazam algorithm [2] is the central reference: it represents each track by salient time–frequency landmarks and hashes pairs of peaks into compact signatures. Its main strength is extremely high precision for exact or near-exact matching, even for short excerpts. Its main limitation is that the representation assumes stable local spectral structure; strong acoustic mismatch, microphone coloration, or reverberation may reduce the number of aligned matches.

Two additional readings reinforced this perspective. The course paper on audio fingerprinting [3] emphasized compact signatures, locality, and lookup efficiency, while the fast music identification reading [4] focused on speed-oriented matching strategies for large music catalogs. These works directly informed the second stage of our system. We did not adopt a purely fingerprint-based architecture, but we used fingerprinting where it is strongest: as a precision-oriented re-ranking stage over a limited candidate list returned by vector retrieval.

In our implementation, the fingerprint stage follows the same overall philosophy but adds an explicit temporal anchor to every hash. This makes offset-histogram alignment possible and therefore reduces accidental collisions between unrelated songs.

2.2 Learned Audio Embeddings

A second family of work focuses on learned audio embeddings, where each audio excerpt is mapped to a dense vector in a latent space. CLAP [5] is the main representative here: it aligns audio and text in a shared embedding space and makes retrieval possible through geometric proximity. This family is attractive because it naturally supports large-scale vector search and moderate invariance to acoustic variation. Its main weakness is that robustness depends strongly on the training distribution and on the domain gap between the reference tracks and the observed query.

Several of the papers studied during the semester fall into this representation-learning family. MuQ [7] and MERT [6] are explicitly music-oriented learned representations. Music2Latent2 [8] and the work on predicting song titles from audio representations [9] both support the broader idea that downstream retrieval quality can benefit from richer latent music descriptors. The reading on audio tokens [10] further emphasized the trend toward more structured learned audio representations. Finally, the deep music transcription paper [16], while not directly an identification paper, highlighted that music can also be represented through higher-level musical structure rather than through exact low-level matching alone.

These readings influenced our Stage 1 design. We implemented MFCC as a classical baseline, CLAP as the main representation, MERT as a comparison model, and MuQ as an implemented but operationally constrained alternative. The practical lesson from the project is that representation quality is not enough by itself: deployability on available hardware and robustness to real microphone captures are equally important. In our experiments, CLAP gave the best overall trade-off between accuracy, robustness, and feasibility on Apple Silicon.

2.3 Large-Scale Similarity Search

A third family concerns large-scale similarity search and vector indexing. FAISS [11] is central in this family: it provides efficient nearest-neighbor search over large vector collections and makes dense audio retrieval operational at scale. Its strength is scalability; its main limitation is that approximate variants introduce a recall-latency trade-off.

The group also studied more system-oriented readings on audio retrieval and vector search [12, 13, 14]. These papers were particularly important for the architectural choices of the project. They stress that persistent vector storage, indexing, and online retrieval are related but distinct concerns. This directly motivated our design choice to use ChromaDB as the persistent source of truth for embeddings, and FAISS as a dedicated search engine rebuilt from that persistent store.

This separation is especially relevant in a Big Data setting. It keeps the ingestion pipeline maintainable, allows method-specific collections and indexes, and lets us rebuild retrieval structures without recomputing the entire dataset.

2.4 Robustness and Signal Enhancement

A fourth family of work concerns robustness and signal enhancement. The speech/audio enhancement reading [15] motivated the idea that degraded observations may be normalized or denoised before retrieval. Its strength is its explicit focus on corrupted signals. Its limitation is that perceptual enhancement does not necessarily optimize retrieval quality.

This distinction turned out to be important in our experiments. We implemented optional spectral denoising in the query pipeline, but our ablations showed that it could degrade the embedding quality rather than help it. In other words, an enhancement method that sounds cleaner is not automatically beneficial for identification. This is why the final system retains loudness normalization and high-pass filtering, but leaves spectral denoising disabled by default.

2.5 Positioning of Our Work

Our project can be positioned between three extremes. A purely fingerprint-based system would emphasize exact local matching but may fail under strong acoustic mismatch. A purely embedding-based system would rely entirely on latent similarity, but may miss the correct track when it is not ranked high enough after Stage 1. A generic ANN engine alone would provide fast search, but no music-specific re-ranking mechanism.

Our approach combines these components in a structured way. Learned embeddings provide high recall over the full database; fingerprinting adds track-level precision on a small candidate set; the hybrid storage architecture separates persistence from fast retrieval. In that sense, our contribution is not the invention of a new isolated algorithm, but the design of a coherent Big Data retrieval pipeline grounded in the actual trade-offs of the codebase, the readings reviewed during the semester, and the hardware constraints under which the project was developed.

3 Problem Formulation

Let $\mathcal{T} = \{t_1, \dots, t_n\}$ denote a database of reference tracks. Each track $t \in \mathcal{T}$ is associated with an audio signal, a set of metadata, a set of overlapping audio segments, and derived representations used for retrieval. At inference time, we observe a query signal q , either from an audio file or from a microphone recording. The objective is to return a ranked list of candidate tracks, ideally with the correct track in first position.

3.1 Segmentation and Representation

Each reference track is segmented into overlapping windows of fixed duration. In our implementation, the window size is 5 seconds and the hop size is 3 seconds, with a minimum retained final segment fraction of 0.8. Let $S(t) = \{s_1, \dots, s_m\}$ denote the set of segments extracted from track t . To each segment s , we associate an embedding $e(s) \in \mathbb{R}^d$, where d depends on the chosen representation. In parallel, we associate to each full track a fingerprint $F(t)$ built from hashes extracted from spectral peaks.

The choice of a 5-second window and a 3-second hop is a compromise between three objectives. First, segments must be long enough to contain musically discriminative information. Second, they must remain short enough to provide local evidence and avoid averaging out the useful structure of the excerpt. Third, the overlap created by a 3-second hop increases temporal coverage and reduces the risk that informative audio events fall exactly at a segment boundary. The downside is a larger number of indexed segments, which increases storage and search cost, but in practice this trade-off was acceptable for the current database size.

3.2 Formalizing Degradations

To model robustness, we distinguish between an ideal clean reference signal x and an observed query q obtained after a sequence of realistic acoustic and transmission degradations. A convenient abstract formulation is:

$$q = C(h((x * r) + \epsilon)),$$

where ϵ denotes additive noise, $*$ denotes convolution, h represents filtering or microphone coloration, and $C(\cdot)$ models codec or compression effects.

Most importantly, r denotes a *room impulse response* (RIR). The RIR characterizes how an acoustic environment transforms the emitted signal before it reaches the microphone: it captures direct-path propagation, early reflections from nearby surfaces, and late reverberation caused by multiple reflections in the room. Therefore, the convolution term $x * r$ explicitly models the impact of room acoustics on the clean signal. In practice, different RIRs correspond to different recording conditions, such as room size, wall materials, source-microphone distance, and microphone placement.

Under this formulation, the observed query q is thus interpreted as a clean signal first altered by room acoustics through the RIR, then contaminated by additive noise, further shaped by channel or microphone filtering, and finally degraded by codec compression. This abstraction is consistent with the simulated degradations implemented in the codebase, namely additive noise, reverberation through RIR-based convolution, band-pass filtering, and codec corruption.

3.3 Stage 1: Segment-Level Vector Retrieval

Retrieval is performed in two stages. In Stage 1, every segment of the query is embedded and compared with the indexed segments of the reference database. For each query segment, the system retrieves the top- k nearest neighbors, with $k = 200$ in the current implementation. Segment-level similarities are then aggregated by track identifier:

$$Score_{\text{faiss}}(t | q) = \sum_{q_i \in S(q)} \sum_{s \in N_{200}(e(q_i)), \text{track}(s)=t} sim(e(q_i), e(s)).$$

The use of a sum as aggregation operator is deliberate. A sum naturally rewards tracks that accumulate evidence over several query segments, which is desirable when the query is longer than a single window. Alternative strategies, such as max pooling, would focus only on the single best local match and would be more sensitive to isolated false positives. Summation therefore acts as a simple and robust multi-segment voting mechanism.

3.4 Stage 2: Fingerprint-Based Re-Ranking

The best track-level candidates are then selected. In the current code, the top $N = 50$ tracks are passed to Stage 2. For these candidates, the system computes a fingerprint similarity score:

$$Score_{fp}(t | q) = \text{fingerprint_similarity}(F(q), F(t)).$$

Importantly, the current implementation does *not* combine Stage 1 and Stage 2 into a single weighted scalar score. Instead, the final ranking is defined by a lexicographic cascade:

$$Key(t | q) = (Score_{fp}(t | q), Score_{faiss}(t | q)),$$

sorted in descending order. The fingerprint score is the primary key and therefore acts as the source of truth whenever it contains a signal; the FAISS score is used only as a secondary fallback key.

The final objective is therefore to maximize the position of the true track in the output ranking. In practice, we evaluate the system using rank, Top-1 accuracy, Stage 1 similarity score, and fingerprint similarity score. This formulation also makes the project constraints explicit: the number of indexed segments is large, the data are heterogeneous, and the hardware environment does not support CUDA-based large-scale inference on NVIDIA GPUs.

4 Solution

4.1 End-to-End Pipeline Overview

Our system is organized in two major phases: offline construction of the reference database and online query processing. The offline phase includes track acquisition, normalization, metadata enrichment, segmentation, embedding computation, fingerprint extraction, storage, and vector index construction. The online phase includes query preprocessing, segmentation, embedding, vector retrieval, fingerprint re-ranking, and result formatting.

This architecture separates expensive offline processing from fast online inference. Such a separation is essential for scalability and reproducibility.

4.2 Building the Reference Database

4.2.1 Source of the Tracks

The ingestion pipeline is designed to accept one or multiple CSV files through the repeated `-csv` option of `manage.py ingest`. In the final project setting, the reference database is built from the full collection of cleaned Spotify chart CSV files available to the group, after title normalization and cross-file deduplication. The evaluation protocol then samples a smaller set of songs and queries from this final reference database in order to keep the report experiments controlled and reproducible.

4.2.2 Title Normalization and Deduplication

Before downloading any track, titles are normalized to reduce duplicates. Variants such as *live*, *radio edit*, or *Taylor's Version* are removed when they are not musically essential. This step is necessary because chart datasets often contain multiple naming variants of the same song, which would artificially bias the database and the index.

4.2.3 Audio Downloading

Tracks are retrieved through YouTube search using the pair *artist + title*. The pipeline then downloads the audio associated with the selected YouTube result and processes it automatically. In practice, this audio often comes from an official music video or another uploaded version of the track, and may therefore differ slightly from the canonical studio recording (e.g., intro/outro segments, live versions, edits, or upload mismatches). In the main ingestion workflow, the system is designed to work mostly in memory rather than maintain a large raw-audio corpus on disk. This makes ingestion lighter and easier to restart.

4.2.4 Incremental Persistence and Validation

Each successfully processed track is saved immediately. This incremental strategy is important because ingestion is long and potentially fragile. It allows the process to resume after interruption without recomputing all previous tracks. In addition, the project provides maintenance and checking scripts to detect incomplete tracks, missing fingerprints, missing embeddings, or metadata inconsistencies.

LISTING 1. Pseudo-code for offline ingestion and indexing

```
Input: one or more CSV files
tracks <- normalize_titles_and_remove_duplicates(CSVs)

for each (artist, title) in tracks:
    audio <- search_and_download_from_youtube(artist, title)
    if audio unavailable:
        continue

    segments <- iter_segments(audio)
    embeddings <- compute_embeddings(segments, method)
    fp <- extract_fingerprint(resample(audio, 22050))

    write embeddings to ChromaDB
    write fingerprint to SQLite
    update metadata parquet

rebuild FAISS index from ChromaDB
```

4.3 Metadata Enrichment

Once tracks are available, their metadata are enriched using external services such as Deezer and MusicBrainz. The target fields include title, artist, album, release date, genre, and cover URL. Metadata serve several purposes: they improve interpretability of results, allow richer result displays, and facilitate later analysis. However, they are not always complete or perfectly consistent. This limitation must be acknowledged explicitly in the report, especially for fields such as genre.

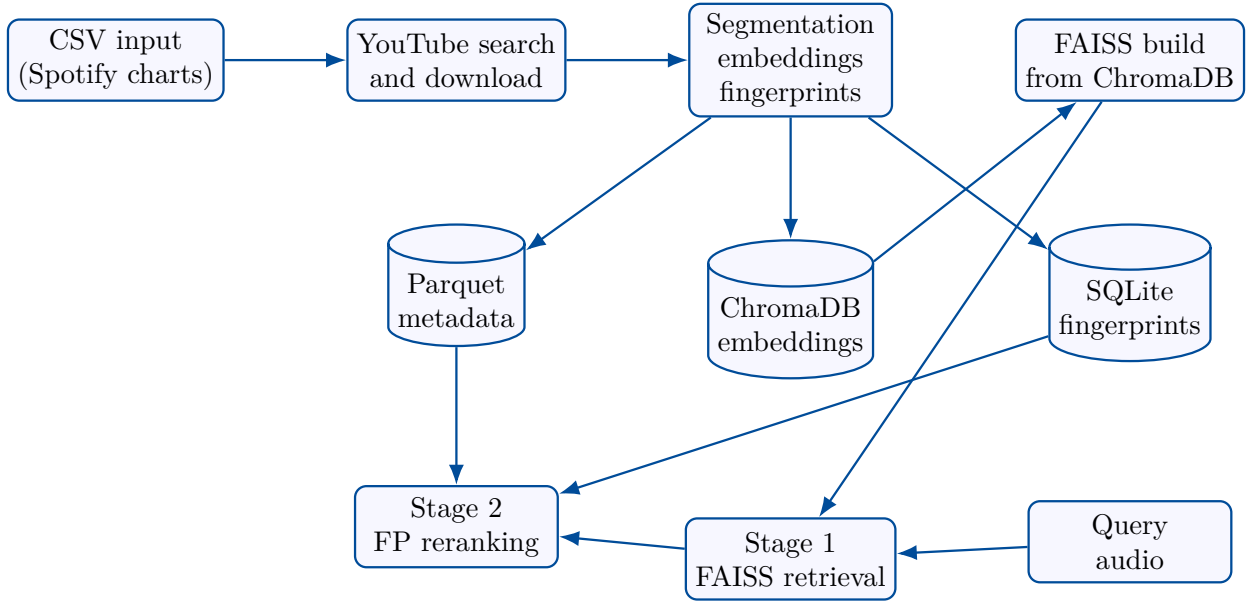


FIGURE 1. Global architecture of the project. Offline processing populates Parquet metadata, ChromaDB embeddings, and SQLite fingerprints; FAISS is rebuilt from ChromaDB and used during Stage 1 retrieval, while SQLite is queried during Stage 2 reranking.

4.4 Audio Processing and Representations

4.4.1 Segmentation

All audio files are segmented into overlapping windows of fixed duration. In our implementation, the default choice is a window of 5 seconds and a hop of 3 seconds. If the remaining tail is at least 0.8 of a full window, it is kept and zero-padded. This overlap increases robustness by ensuring that informative parts of the track are unlikely to fall exactly at a segment boundary.

4.4.2 Sampling Rates and Embedding Dimensions

Different representations operate at different sampling rates and produce embeddings of different dimensionalities. Table 1 summarizes the concrete values used in the code.

TABLE 1. Representations used in the project.

Method	Input SR	Dim.	Role
MFCC	22 050 Hz	40	baseline
CLAP	48 000 Hz	512	main method
MERT	internal resample to 24 000 Hz	768	comparison
MuQ	24 000 Hz	1024	implemented, not retained

The MFCC embedding is built by concatenating the mean and standard deviation of 20 coefficients, leading to a 40-dimensional vector. CLAP is the main representation

used in the final pipeline. MERT remains implemented in the code and can be evaluated, but it is not the active default method. MuQ is also implemented, but was not retained in the final experiments due to hardware limitations.

4.4.3 MFCC Baseline

MFCC statistics are used as a lightweight baseline. Let $m_{k,t}$ denote the k -th MFCC coefficient at frame t . The embedding is formed by concatenating the mean and standard deviation of each coefficient across time:

$$\mu_k = \frac{1}{T} \sum_{t=1}^T m_{k,t}, \quad \sigma_k = \sqrt{\frac{1}{T} \sum_{t=1}^T (m_{k,t} - \mu_k)^2},$$

and

$$e_{\text{MFCC}}(s) = [\mu_1, \dots, \mu_{20}, \sigma_1, \dots, \sigma_{20}] \in \mathbb{R}^{40}.$$

This representation is classical, simple to compute, and provides an interpretable benchmark against which learned embeddings can be compared. Its main drawback is that it remains less expressive than modern latent audio representations.

4.4.4 CLAP Embeddings

CLAP is the main embedding used in the final pipeline. It provides a dense vector representation of each segment and serves as the basis of Stage 1 vector retrieval. In our experiments, CLAP offered the best trade-off between practicality, robustness, and retrieval quality. We also compared different CLAP configurations, including `clap-htsat-unfused` and `larger_clap_music`.

At the representation-learning level, CLAP is based on a contrastive objective aligning audio and text embeddings. In simplified form, for a batch of matching audio/text pairs (a_i, t_i) , the objective can be written as

$$\mathcal{L}_{\text{CLAP}} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(\langle a_i, t_i \rangle / \tau)}{\sum_{j=1}^B \exp(\langle a_i, t_j \rangle / \tau)},$$

with temperature parameter τ . In our project, we do not retrain CLAP; we use it as a pretrained feature extractor.

4.4.5 MERT as an Alternative Learned Representation

MERT is implemented in the codebase and can be selected as an embedding method. In our project, it was evaluated as a comparison model rather than retained as the default method. In our experimental setup, it behaved correctly on clean material but remained less robust than CLAP on the difficult real microphone query.

4.4.6 Why MuQ Was Not Retained

MuQ is implemented in the repository, but it was not retained in the final solution and in the main comparative experiments. The practical issue is not that MuQ is absent from the code, but that the project hardware does not include an NVIDIA GPU and therefore cannot benefit from CUDA acceleration. MuQ can technically run on CPU, but this is not operationally realistic for large-scale experimentation in our setting. In addition, MuQ does not benefit from the same MPS support assumptions as the rest of the pipeline, which further limits its usability on Apple Silicon for our workload.

4.5 Fingerprinting Stage

In parallel with embeddings, we compute a Shazam-inspired fingerprint for each reference track. The signal is transformed into a spectrogram using STFT with $n_fft = 2048$ and $hop_length = 512$. Salient local peaks are extracted using a two-dimensional maximum filter with neighborhood size $(20, 15)$, that is, 20 frequency bins by 15 time frames. Peaks are retained above the 80th percentile of the spectrogram magnitude.

Pairs of peaks are then converted into hashes. In the current implementation, each hash has the form

$$h = (f_1, f_2, \Delta t, t_1), \quad \Delta t = t_2 - t_1,$$

where t_1 is a temporal anchor. The fan-out is limited to 5 pairs per anchor peak, and only temporal gaps between 3 and 50 frames are kept:

$$3 \leq \Delta t \leq 50.$$

This design makes it possible to compute an offset histogram between the query and a candidate track. The final fingerprint similarity is

$$\text{Score}_{\text{fp}}(q, t) = \frac{\max_{\delta} H_{q,t}(\delta)}{|F(q)|},$$

where $H_{q,t}(\delta)$ is the histogram count of matching hashes at temporal offset δ .

A crucial detail of the code is that fingerprinting always operates at 22 050 Hz, independently of the embedding method used for Stage 1. If the query was processed at another sampling rate for embeddings, it is resampled back to 22 050 Hz before fingerprint extraction.

The current hyperparameters were chosen to keep a balance between robustness and sparsity. A larger neighborhood would produce too few peaks; a smaller one would flood the representation with unstable local maxima. The fan-out of 5 limits combinatorial explosion in the number of hashes, while the Δt bounds discard pairs that are too close to be uninformative or too far to remain reliable.

LISTING 2. Pseudo-code for fingerprint extraction

```

Input: waveform x at 22050 Hz
S <- abs(STFT(x, n_fft=2048, hop=512))
P <- local_maxima(S, neighborhood=(20,15))
P <- keep peaks above 80th percentile

H <- empty set
for each anchor peak (t1, f1) in P:
  count <- 0
  for each later peak (t2, f2):
    dt <- t2 - t1
    if 3 <= dt <= 50:
      add (f1, f2, dt, t1) to H
      count <- count + 1
      if count == 5:
        break
    if dt > 50:
      break
return H

```

4.6 Data Management and Storage Architecture

The data architecture is one of the key contributions of the project. The system manipulates at least three different kinds of data: dense vectors, discrete fingerprints, and tabular metadata. These objects do not share the same structure or access patterns, so a single storage system would not be appropriate.

4.6.1 Data Integrity and Operability

Because the project relies on several persistent stores (ChromaDB, FAISS, SQLite, and Parquet), operational consistency is an important part of the solution. The repository therefore includes dedicated maintenance commands to verify and restore data integrity.

The `check` command detects critical inconsistencies such as malformed embeddings, ChromaDB/FAISS desynchronization, orphan segments, incomplete embeddings, and missing fingerprints. The `rebuild` command reconstructs fingerprints and/or FAISS indexes after a purge, a crash, or a parameter change. Together, these tools make the pipeline maintainable and reproducible over long ingestion runs, which is an important Big Data engineering aspect of the project rather than a secondary convenience.

4.6.2 ChromaDB for Persistent Embeddings

ChromaDB stores the embeddings of all reference segments together with their metadata, such as `track_id` and time offset. It serves as the persistent source of truth. This is useful because embeddings may need to be deleted, recomputed, or reindexed track by track. A distinct collection key is derived from the method and model name, which avoids mixing embeddings from incompatible latent spaces.

4.6.3 FAISS for Fast Retrieval

FAISS is used as the retrieval engine at inference time. The index is rebuilt from the vectors stored in ChromaDB. This separation between persistence and retrieval is beneficial: ChromaDB provides stable storage, while FAISS is optimized for nearest-neighbor search. In the current configuration, the index retrieves 200 candidate segments per query segment.

The codebase implements three FAISS index families: `flat`, `hnsw`, and `ivf`. However, the experiments reported in this document use the `flat` configuration in order to avoid conflating audio-model effects with ANN approximation effects. A systematic recall/latency comparison between `flat`, `hnsw`, and `ivf` is therefore left as future work.

4.6.4 SQLite for Fingerprints

Fingerprints are stored in SQLite, with one row per track. This choice is natural because Stage 2 requires only targeted access to a limited set of candidate tracks rather than a global approximate nearest-neighbor search. In the current implementation, only the top 50 tracks from Stage 1 are passed to the fingerprint stage.

4.6.5 Why This Architecture Fits the Data and Queries

This hybrid architecture is well suited to the project because each data type is matched with the query pattern it supports:

- embeddings require large-scale similarity search;
- fingerprints require direct lookup by track identifier;
- metadata require structured tabular access.

Using different storage backends for these different access patterns is therefore a justified Big Data design choice rather than an arbitrary implementation detail.

4.7 Query Pipeline

A query may be either an uploaded audio file or a microphone recording. In both cases, the system first preprocesses the signal, then segments it, computes one embedding per segment, and performs vector retrieval.

Although the primary usage mode of the project remains the command-line interface, we also implemented a web application as an additional deliverable. This web layer turns the identification pipeline into a directly usable end-user system: it accepts either uploaded audio files or microphone recordings, calls the same backend identification logic, and returns ranked results enriched with metadata, artwork, streaming links, and simple recommendations. This makes the project not only reproducible from the CLI, but also demonstrable as a complete application.

The preprocessing stage currently contains four ordered operations:

1. a 4th-order Butterworth high-pass filter at 80 Hz;
2. an optional non-stationary spectral denoising step controlled by `OPT_QUERY_DENOISE` (disabled by default);
3. loudness normalization to -14 LUFS;
4. peak normalization to 0.95.

The denoising step is part of the codebase and was evaluated experimentally, even though it is disabled in the final default configuration. The report should therefore describe it as an implemented but deactivated option rather than omit it from the method.

The query sampling rate depends on the selected embedding method. In `identify_track`, CLAP queries are loaded at 48 000 Hz; MuQ queries are loaded at 24 000 Hz; the default loader uses 22 050 Hz otherwise. In the case of MERT, the embedding function then performs an internal

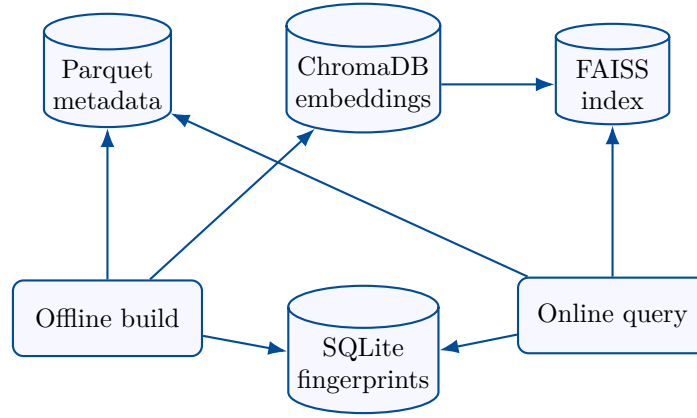


FIGURE 2. Hybrid storage architecture used in the project.

resampling to 24 000 Hz. After Stage 1, the waveform is always resampled to 22 050 Hz for fingerprint extraction.

In Stage 1, each query segment is embedded and searched in the FAISS index. The nearest neighbors are retrieved, and their scores are aggregated by `track_id`. This produces a track-level ranking based on vector similarity.

In Stage 2, only the top 50 Stage 1 candidates are kept. The fingerprint of the query is compared with the fingerprint of each candidate. The current code does *not* merge both scores into a weighted scalar. Instead, the final list is sorted lexicographically:

1. primary key: fingerprint score (descending);
2. secondary key: FAISS score (descending).

This means that fingerprinting is the source of truth whenever it produces a non-zero signal, while FAISS only resolves ties or acts as a fallback when all fingerprint scores are zero.

LISTING 3. Pseudo-code for the online query pipeline

```

Input: query audio q, method m
q <- preprocess_query(q)
segments <- iter_segments(q)
scores_faiss <- empty map
for each segment s in segments:
    emb <- embed_segment(s, method=m)
    nn <- FAISS.search(emb, k=200)
    accumulate FAISS scores by track_id

candidates <- top 50 tracks by FAISS score
q_fp <- extract_fingerprint(resample(q, 22050))
for each track t in candidates:
    fp_t <- load fingerprint from SQLite
    score_fp[t] <- fingerprint_similarity(q_fp, fp_t)

sort candidates by (score_fp, score_faiss) descending
return top 10 results
  
```

4.8 Robustness Through RIR Augmentation

A central difficulty of the project was the performance gap between clean reference material and real microphone queries. To reduce this mismatch, we introduced room impulse response (RIR) augmentation. A room impulse response simulates the acoustic response of an environment; convolving a studio recording with an RIR therefore produces a more realistic room-like version of the same song.

In practice, several RIR-based variants of the reference tracks are generated, embeddings are computed for those degraded variants, and they are inserted into the database under the same `track_id`. In the current configuration, the project uses 5 RIRs per track by default (`RIR_N = 5`), with either synthetic or MIT-based sources. The objective is to reduce the domain shift between clean studio tracks and real acoustic observations.

This strategy proved complementary to query preprocessing. Preprocessing stabilizes the observed signal, while RIR augmentation enriches the reference database with more acoustically realistic variants. Together, these two mechanisms produced the most important gains on the difficult microphone case.

5 Experiments

5.1 Experimental Setup

5.1.1 Hardware and Software Environment

The experiments were conducted on an Apple Silicon machine using CPU/MPS acceleration when available. No NVIDIA GPU was available, and therefore no CUDA-based setup was used. This hardware constraint influenced our methodological choices, especially the exclusion of MuQ from the final system.

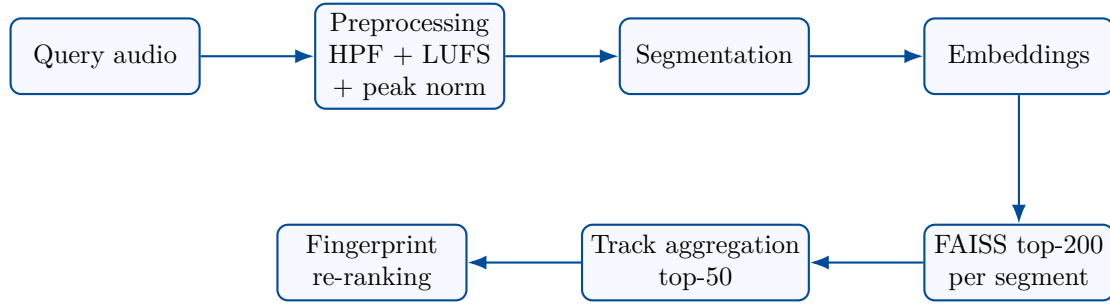


FIGURE 3. Online query pipeline from raw audio to final ranking.

5.1.2 Dataset Statistics and CSV Usage

The ingestion CLI supports one or multiple CSV files. For the final reference corpus targeted by the project, ingesting all available Spotify chart CSV files, normalizing titles, and merging duplicates across files yields a total of 4 257 distinct tracks from 1 639 artists. This largely satisfies the course requirement of more than 800 songs.

At the representation level, each reference track is split into overlapping 5-second segments with a 3-second hop. Using the observed average indexing density of the project, the final reference corpus corresponds to approximately 292 876 indexed segments for the standard indexes. With CLAP and RIR augmentation, this scales to about 1 757 258 stored vectors. In parallel, 4 257 fingerprints are stored in SQLite, one per track.

TABLE 2. Main statistics of the full reference database.

Quantity	Value
Final reference tracks	4 257
Artists	1 639
Standard indexed segments	$\approx 292\,876$
CLAP + RIR vectors	$\approx 1\,757\,258$
Fingerprints in SQLite	4 257
Course requirement	> 800 tracks

5.1.3 Evaluation Metrics

We evaluate the system using:

- the rank of the target song in the final pipeline;
- Top-1 and Top-5 accuracy;
- Mean Reciprocal Rank (MRR);
- average query latency;
- the Stage 1 rank and FAISS score;
- the Stage 2 fingerprint score.

5.1.4 Index Type Used in Reported Experiments

Although the codebase implements three FAISS index families (`flat`, `hnsf`, and `ivf`), all the experiments reported here use the `flat` index. This ensures that the measured differences come from the audio representations, preprocessing steps, and data augmentation choices rather than from ANN approximation errors. A systematic runtime/recall comparison of `flat`, `hnsf`, and `ivf` is therefore left for future work.

5.2 Overall Evaluation

The evaluation suite generated by `python manage.py eval` contains 59 real queries drawn from 8 songs listed in the manifest. It includes 24 studio excerpts of 5 s, 15 s, and 30 s, and 35 microphone recordings collected under different conditions (distance and concurrent speech). All results reported in this section correspond to the current CLAP-based configuration.

On the full 59-query suite, Stage 1 alone reaches 22.0% Top-1 accuracy (13/59) and 62.7% Top-5 accuracy (37/59). After Stage 2 re-ranking, final performance rises to 72.9% Top-1 accuracy (43/59) and 83.1% Top-5 accuracy (49/59). The mean rank reported by the evaluation summary decreases from 3.8 at Stage 1 to 1.12 after the final ranking. The average query latency over the suite is 23.3 s.

These results show that the full pipeline is substantially stronger than Stage 1 alone. They also indicate that the current evaluation is no longer a small synthetic benchmark, but a real-query protocol covering both clean reference excerpts and realistic microphone captures.

5.3 Studio Versus Microphone Queries

The evaluation confirms that studio excerpts remain easier than microphone recordings. On studio queries, the system reaches 79.2% Top-1 accuracy (19/24) and 91.7% Top-5 accuracy (22/24). On microphone queries, performance decreases to 68.6% Top-1 (24/35) and 77.1% Top-5 (27/35).

This gap is also visible in the intermediate scores. The mean FAISS score is 31.67 for studio queries but only 11.62 for microphone queries, which confirms a substantial domain shift between clean reference material and real acoustic captures. Nevertheless, the final ranking remains strong on many microphone recordings thanks to Stage 2 re-ranking.

5.4 Effect of Query Duration

The duration analysis was performed on the studio excerpts only. Longer excerpts clearly improve Stage 1 retrieval. Stage 1 Top-1 accuracy rises from 12.5% (1/8) for 5 s excerpts to 62.5% (5/8) for 15 s and 75.0% (6/8) for 30 s. The mean Stage 1 rank also improves from 4.86 at 5 s to 1.25 at 30 s.

After the final re-ranking, the effect of duration remains visible but becomes less pronounced: final Top-1 accuracy is 75.0% (6/8) for both 5 s and 15 s excerpts, and 87.5% (7/8) for 30 s. Final Top-5 accuracy reaches 100.0% for 30 s excerpts. This suggests that longer queries mainly help by making the correct track easier to retrieve at Stage 1.

5.5 Stage 1 Versus Stage 2

The comparison between Stage 1 and the final pipeline is one of the clearest results of the current evaluation. Over all 59 queries, Stage 2 improves the rank in 69.4% of the cases, leaves it unchanged in 30.6%, and never degrades it. Final Top-1 accuracy increases from 22.0% at Stage 1 to 72.9% after re-ranking.

The gain is even larger on microphone queries. On this subset, Stage 1 Top-1 accuracy is only 2.9% (1/35), whereas the final Top-1 accuracy reaches 68.6% (24/35). In this subset, 92.6% of the queries are improved by Stage 2 and none are degraded. This confirms that fingerprint-based re-ranking is essential in the current pipeline.

However, Stage 2 does not solve every failure. Ten queries remain not found in the final output. In particular, some microphone recordings of *As It Was* and *TQG* still fail completely, while *Calm Down* is repeatedly retrieved at rank 2 rather than rank 1. This shows that Stage 2 is highly effective when the correct track is sufficiently represented among the candidates, but cannot fully compensate for all mismatches between query and reference data.

5.6 Microphone Condition Analysis

The microphone-condition analysis shows that performance depends on the recording setup, although the subsets are small and should be interpreted cautiously. The best result in the current sample is obtained for *normal distance + speech*, with 87.5% Top-1 accuracy (7/8) and 100.0% Top-5 accuracy (8/8). By contrast, *close + clean* and *close + speech* both reach 62.5% Top-1 accuracy (5/8). The *far + clean* subset reaches 66.7% Top-1 accuracy, but it contains only 3 queries.

An interesting observation is that the presence of concurrent speech is not uniformly harmful in the current sample. The system performs better on *normal + speech* than on several “clean” microphone conditions. This suggests that query difficulty is not determined by a single factor, but by the interaction between distance, room acoustics, source level, and competing sounds.

5.7 Complementary Robustness Ablations

The 59-query evaluation suite is the main protocol used in this report because it measures end-to-end performance on real queries across several songs and microphone conditions. However, the repository also contains a complementary benchmark log dedicated to controlled ablations on robustness [17]. This second protocol is narrower in scope, but it is useful to isolate the effect of preprocessing, augmentation, and model choices.

5.7.1 Query Preprocessing Ablation

One of the clearest ablation results concerns query preprocessing. On the difficult real microphone recording used as a diagnostic case in the benchmark log, adding only loudness normalization and an 80 Hz high-pass filter moved the correct song from rank #64 to rank #6 in Stage 1. By contrast, adding non-stationary spectral denoising degraded the same case from rank #6 to rank #36. This confirms two practical points: level mismatch was a major issue for CLAP-based retrieval, and denoising that sounds perceptually reasonable can still perturb the embedding space.

5.7.2 RIR Augmentation

RIR augmentation is one of the strongest robustness improvements retained in the final project. The key point is to distinguish it from simpler synthetic degradations such as additive white noise, which did not reproduce the real microphone mismatch well enough. In contrast, augmenting the reference database with room-impulse-response-transformed versions of the tracks proved genuinely useful.

The intuition is that the difficult queries of the project are not only noisy: they are also affected by room acoustics, source-microphone geometry, and device coloration. Convolution of clean reference tracks with diverse room impulse responses makes the indexed vectors acoustically closer to real room-recorded queries. In the implemented pipeline, these RIR-transformed variants are embedded and inserted into ChromaDB under the same `track_id`, so they enrich Stage 1 recall without changing the final track identity.

The controlled ablation confirms this effect. On the diagnostic real microphone case used throughout the robustness log, after query preprocessing was fixed, removing RIR-augmented vectors lowered the target from rank #6 to rank #18, while the FAISS score dropped from 22.68 to 14.42. Conversely, keeping the RIR-augmented database restored the target to rank #6. On the complementary 14-case robustness benchmark, augmentation raised Top-1 accuracy from 57.1% to 85.7%, with the largest gains on reverberant and combined degradations. Therefore, the final interpretation is not that “any artificial degradation helps”, but rather that acoustically plausible reverberation augmentation helps because it reduces the domain gap between clean downloaded references and real room-recorded queries. The corresponding visual summary is shown in Appendix Figure 4.

5.7.3 Model Comparison and Heuristic Variants

The project also includes targeted model comparisons. On the difficult real microphone case used in the benchmark notes, `clap-htsat-unfused` ranked the target at #70 during Stage 1, whereas `larger_clap_music` ranked it at #186 and MERT at #212. At the same time, MERT still identified the clean studio excerpt correctly at rank 1. This suggests that a music-specialized representation is not automatically more robust to the exact acoustic mismatch created by the microphone setup used in our experiments.

Several fingerprint heuristics were also tested, including adaptive thresholding and larger query fan-out. In the benchmark log, these variants either failed to improve performance or introduced regressions. This empirical result supports the final design choice of keeping the fingerprint stage simple, stable, and primarily focused on re-ranking rather than aggressive heuristic tuning.

5.8 Discussion

Taken together, the two experimental protocols support three main conclusions. First, the principal weakness of the system lies in the mismatch between clean reference audio and real microphone queries, as shown by the much lower Stage 1 scores on microphone data. Second, Stage 2 re-ranking is crucial: on the real-query suite it converts a weak Stage 1 baseline into a much stronger final pipeline without observed regressions. Third, the complementary ablations show that robustness gains do not come from a single trick, but from the interaction between level-aware preprocessing and acoustically realistic augmentation of the reference database.

At the same time, the results also reveal persistent track-specific failures. Some songs, such as *Daylight*, *Tattoo*, *Flowers*, and *See You Again*, are identified very reliably across the available queries, whereas others remain problematic. This suggests that future work should focus not only on global robustness, but also on reference-query alignment for difficult tracks and on extending the real-query evaluation beyond the current eight-song manifest.

5.9 Evaluation Scope and Current Limits

The present `manage.py eval` outputs support claims about overall accuracy, studio-versus-microphone robustness, query duration, Stage 1 versus Stage 2 behavior, and microphone recording conditions. The complementary benchmark log supports additional claims about preprocessing ablations, synthetic degradations, RIR augmentation, and targeted model comparisons. Nevertheless, these secondary experiments remain narrower than the main real-query suite: they are diagnostic experiments centered on a limited set of songs and degradations, not global estimates over the full 4 257-track catalog.

6 Conclusion

This project presented a complete Big Data pipeline for music identification from short audio excerpts. The system integrates track acquisition, metadata enrichment, segmentation, embedding computation, fingerprint extraction, hybrid storage, vector indexing, retrieval, and re-ranking. Beyond the implementation of an identification engine, the project emphasized the importance of end-to-end system design.

Our final architecture combines ChromaDB for persistent embeddings, FAISS for scalable nearest-neighbor search, SQLite for track-level fingerprints, and Parquet for metadata. This hybrid design matches the structure of the data and the query patterns of the system, and therefore constitutes a justified data-management choice.

From a methodological perspective, the best final configuration relies on CLAP embeddings, a fingerprint-based re-ranking stage, careful query preprocessing, and RIR-based augmentation of the reference database. Quantitatively, on the main 59-query real evaluation suite, Stage 1 alone reaches 22.0% Top-1 accuracy and the final pipeline reaches 72.9%. On the complementary robustness benchmark used for ablations, the baseline reached 57.1% Top-1 accuracy, while augmentation raised this figure to 85.7%. On the difficult real microphone case used as a diagnostic benchmark, preprocessing moved the target from rank #64 to rank #6, and RIR support improved a matched comparison from rank #18 to rank #6.

The project nevertheless has several limitations. Even though the final reference corpus of 4 257 tracks largely satisfies the course requirement, it remains relatively small compared with industrial systems. Some metadata are incomplete, and the microphone case remains sensitive to the acoustic environment and to track-specific mismatches between the downloaded reference audio and the recorded excerpt. In addition, although the code implements several FAISS index types, the present report evaluates only the exact `flat` setting.

Future work should address scalability more explicitly. With a database 10x or 1000x larger, the number of indexed segments would increase dramatically, making index choice, memory footprint, and ANN recall/latency trade-offs much more critical. Future improvements could therefore include broader FAISS comparisons, larger and more diverse reference collections, more real microphone recordings, richer RIR sets, and further evaluation of music-oriented embedding models under realistic deployment constraints.

A Use of Artificial Intelligence

LLM-based tools were used in a limited and supervised way during the project, mainly as support tools rather than as autonomous builders of the system.

First, they were used during the literature-review phase to clarify technical concepts, compare families of methods already selected by the group, and help summarize differences between fingerprinting, learned embeddings, vector search, and robustness techniques. This support was used to speed up understanding of papers, not to replace paper reading.

Second, they were used during development as debugging assistants on specific issues: understanding error messages, checking the meaning of library behaviors, discussing implementation alternatives, and helping reason about design trade-offs. Typical examples include questions related to model compatibility on Apple Silicon, preprocessing choices, storage organization, and failure analysis of robustness experiments. In all such cases, the code was written, reviewed, modified, and validated by the authors.

Third, they were used during report writing only for language support: reformulating some sentences in clearer academic English, improving transitions between paragraphs, and polishing phrasing. They were not used to generate the experimental results, fabricate measurements, invent references, or automatically write the full report without verification.

Conversely, the following elements were produced and validated by the group itself: dataset construction, ingestion scripts, retrieval pipeline implementation, storage design, evaluation scripts, benchmark execution, plots, result interpretation, and final methodological choices. The authors remain fully responsible for the code, experiments, analysis, and conclusions presented in this report.

B References

References

- [1] Apple. Shazam hits 100 billion song recognitions. Apple Newsroom, 20 November 2024.
- [2] Avery Wang. An Industrial-Strength Audio Search Algorithm. In *Proceedings of the 4th International Conference on Music Information Retrieval (ISMIR)*, 2003.
- [3] Pedro Cano, Eloi Batlle, Ton Kalker, and Jaap Haitsma. A Review of Audio Fingerprinting. *Journal of VLSI Signal Processing Systems*, 41:271–284, 2005.
- [4] Guang-Ho Cha. Indexing and Search for Fast Music Identification. In *CD-ARES 2012*, pages 259–271, 2012.
- [5] Benjamin Elizalde, Soham Deshmukh, Mahmoud Al Ismail, and Huaming Wang. CLAP: Learning Audio Concepts from Natural Language Supervision. In *ICASSP 2023*, pages 1–5, 2023.
- [6] Yizhi Li, Ruibin Yuan, Ge Zhang, Yinghao Ma, Xingran Chen, Hanzhi Yin, Chenghua Lin, Anton Ragni, Emmanouil Benetos, Norbert Gyenge, Roger Dannenberg, Ruibo Liu, Wenhua Chen, Gus Xia, Yemin Shi, Wenhao Huang, Yike Guo, and Jie Fu. MERT: Acoustic Music Understanding Model with Large-Scale Self-supervised Training. *CoRR*, abs/2306.00107, 2023.
- [7] Haina Zhu, Yizhi Zhou, Hangting Chen, Jianwei Yu, Ziyang Ma, Rongzhi Gu, Wei Tan, and Xie Chen. MuQ: Self-Supervised Music Representation Learning with Mel Residual Vector Quantization. *CoRR*, abs/2501.01108, 2025.
- [8] Marco Pasini, Stefan Lattner, and George Fazekas. Music2Latent2: Audio Compression with Summary Embeddings and Autoregressive Decoding. *CoRR*, abs/2501.17578, 2025.
- [9] Avi Bleiweiss. Predicting a Song Title from Audio Embeddings on a Pretrained Image-captioning Network. In *Proceedings of ICAART 2020*, pages 483–493, 2020.
- [10] Pooneh Mousavi, Gallil Maimon, Adel Moumen, Darius Petermann, Jiatong Shi, Haibin Wu, Haici Yang, Anastasia Kuznetsova, Artem Ploujnikov, Ricardo Marxer, Bhuvana Ramabhadran, Benjamin Elizalde, Loren Lugosch, Jinyu Li, Cem Subakan, Phil Woodland, Minje Kim, Hung-yi Lee, Shinji Watanabe, Yossi Adi, and Mirco Ravanelli. Discrete Audio Tokens: More Than a Survey! *CoRR*, abs/2506.10274, 2025.
- [11] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.
- [12] Benjamin Elizalde, Shuayb Zarar, and Bhiksha Raj. Cross Modal Audio Search and Retrieval with Joint Embeddings Based on Text and Audio. In *ICASSP 2019*, pages 4095–4099, 2019.
- [13] Leonardo Kuffo, Elena Krippner, and Peter Boncz. PDX: A Data Layout for Vector Similarity Search. *Proceedings of the ACM on Management of Data*, 3(3):196:1–196:26, 2025.
- [14] Dominik Schnitzer, Arthur Flexer, and Gerhard Widmer. A Fast Audio Similarity Retrieval Method for Millions of Music Tracks. *Multimedia Tools and Applications*, 58(1):23–40, 2012.
- [15] Szymon Drgas. A Survey on Low-Latency DNN-Based Speech Enhancement. *Sensors*, 23(3):1380, 2023.
- [16] Nicolas Boulanger-Lewandowski, Yoshua Bengio, and Pascal Vincent. Modeling Temporal Dependencies in High-Dimensional Sequences: Application to Polyphonic Music Generation and Transcription. In *ICML 2012*, 2012.
- [17] Project benchmark log. `results/EXPERIMENTS.md`, repository version used as supporting evidence for robustness ablations.

C Basic Usage and Reproducibility

This appendix gathers the main commands needed to reproduce the system and test it in practice, either from the command line with a studio excerpt or through the web interface.

LISTING 4. Main project commands

```
# Full pipeline on all CSV files available in data/kaggle/data/
python manage.py build

# Full pipeline on selected CSV files only
python manage.py build \
  --csv data/kaggle/data/spotify-streaming-top-50-world.csv \
  --csv data/kaggle/data/spotify-streaming-top-50-france.csv

# Rebuild fingerprints and FAISS indexes
python manage.py rebuild --what all

# Check database consistency
python manage.py check --details

# Download a studio test excerpt from YouTube
python manage.py download-test "Miley Cyrus Flowers" \
  --duration 15 --position middle

# Identify one audio file from the command line
python manage.py identify \
  data/raw/reference_clips/Miley_Cyrus-Flowers-middle_15s.mp3 \
  --top 10 --detailed

# Launch the web interface
python manage.py webapp
```

D User Interfaces

The project exposes two main usage modes. The primary interface is the command-line interface, which supports ingestion, augmentation, checking, rebuilding, evaluation, and direct audio identification. In addition, we implemented a web application as an end-user-oriented deliverable. It can be launched through the dedicated startup script, accepts either microphone recordings or uploaded audio files, and displays ranked matches enriched with metadata, artwork, streaming links, and simple genre-based recommendations. This additional interface makes the system easier to demonstrate and use beyond the CLI workflow.

At the API level, the FastAPI backend exposes three main routes: `/api/identify` for audio identification, `/api/config` for user-interface configuration, and `/api/health` for service monitoring. In development mode, the frontend is served through Vite; in production mode, the built React application is served by FastAPI. Full-width screenshots of the main web interface states are provided in Appendix Figure 5.

E Visual Appendix

This appendix gathers the main visual material of the report in full-width format for readability. Only this final section switches out of the two-column layout, so the rest of the document keeps the compact report format while the visuals remain easy to inspect in the final PDF.

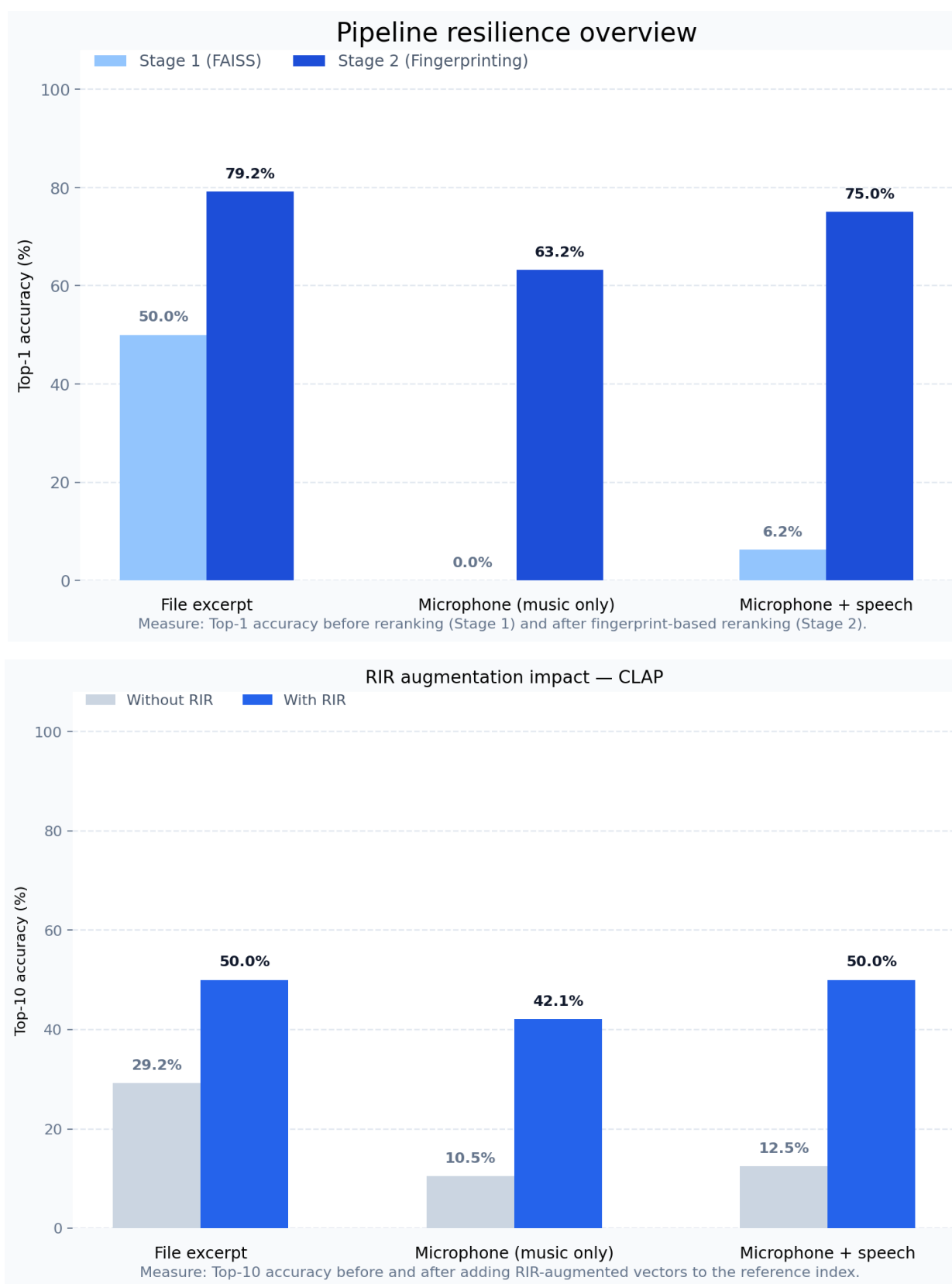


FIGURE 4. Full-width appendix plate for the experimental visual summaries. The top panel shows the main resilience overview of the retrieval pipeline; the bottom panel shows the dedicated RIR analysis overview.

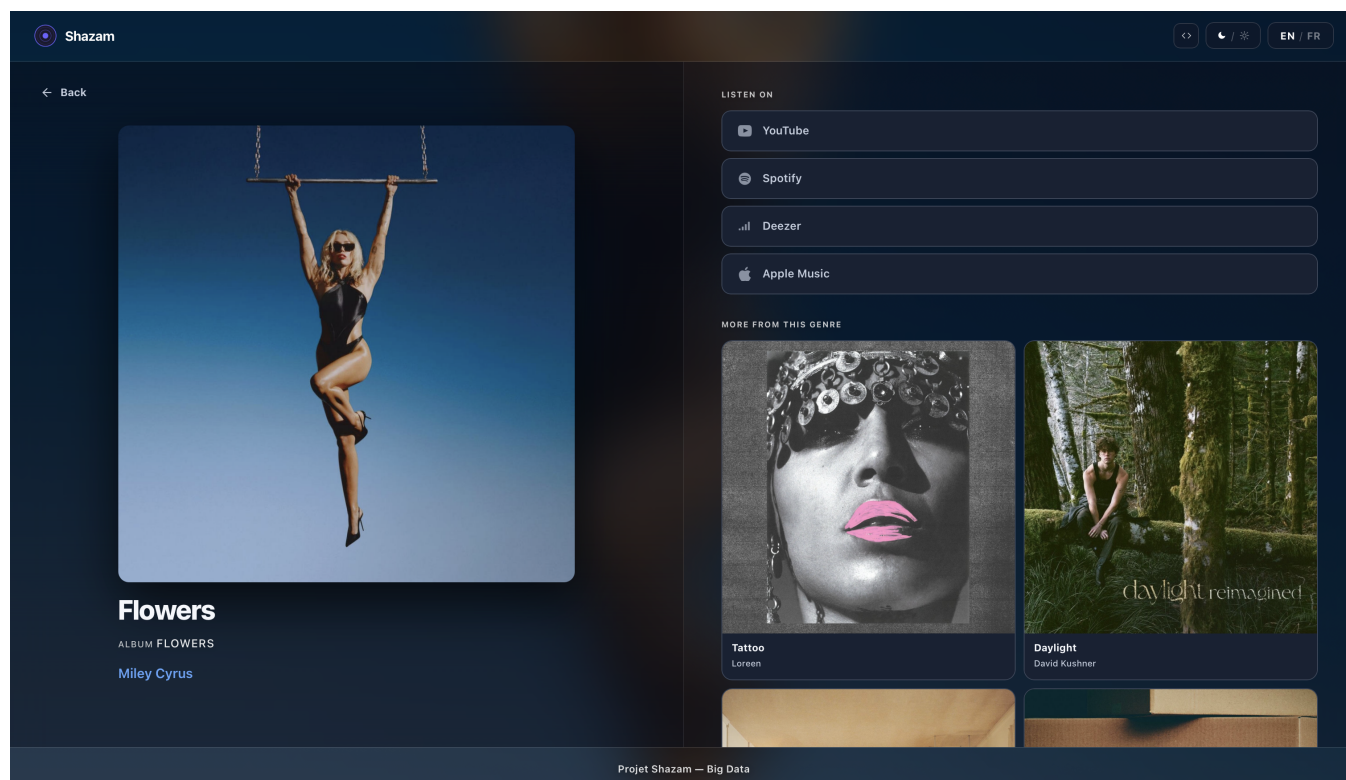
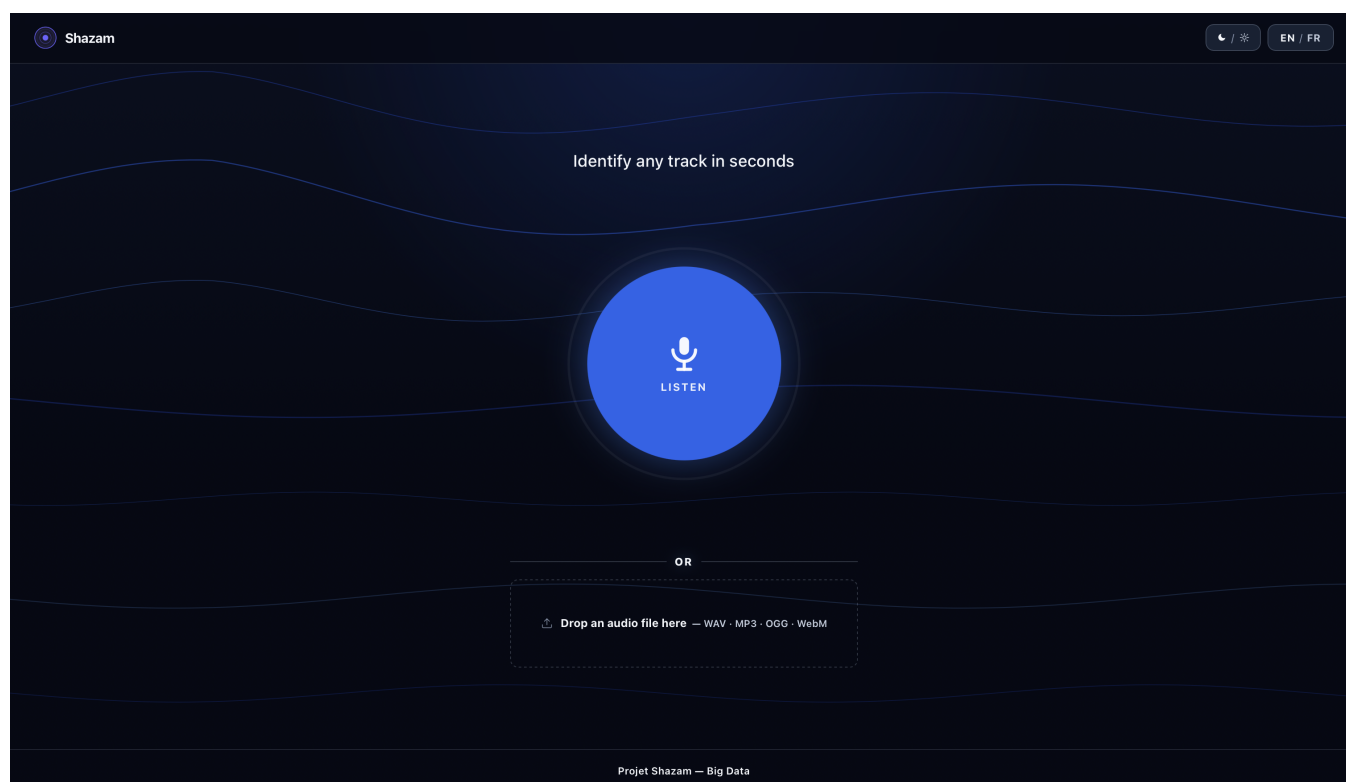


FIGURE 5. Full-width appendix plate for the web interface. The top panel shows the application start page with recording and upload entry points; the bottom panel shows the result page with metadata and links.